Lecture 11 · Wed 11/4/2026

CS160 Compilers — Fall 2026

Operational Semantics, and Adding Lists.

Yu Feng

Two kinds of rules.

	Types (Lecture 9)	Values (today)
expression	$\Gamma \vdash e : T$	$G, E, S \vdash e : v, S'$
statement	$\Gamma, R \vdash s$	$G, E, S \vdash s : S', R'$

operational semantics
rules for how a program runs

G, E
where global and local names live

S
the **store**: what each location holds, plus the output

v
the value

S'
the store afterwards

R'
the value a `return` carries out, or `None`

Type rules say what *kind* of value. Value rules say *which* value, and what changes.

Constants, variables, assignment.

$$\frac{}{G, E, S \vdash n : n, S}\ \text{E-Const} \qquad \frac{}{G, E, S \vdash x : S(E(x)), S}\ \text{E-Var}$$

$$\frac{G, E, S \vdash e : v, S_1}{G, E, S \vdash x = e : S_1[E(x) \mapsto v], \text{None}}\ \text{S-Assign}$$

E(x)
the location of x: in E if local, else in G

S[ℓ ↦ v]
S with location ℓ now holding v

A variable means what its location holds. Assignment changes that location.

Operators.

$$\frac{G, E, S \vdash e_1 : v_1, S_1 \qquad G, E, S_1 \vdash e_2 : v_2, S_2}{G, E, S \vdash e_1 + e_2 : \mathrm{wrap}(v_1 + v_2), S_2}\ \text{E-Add}$$

(E-Mul: the same, with ×)

$$\frac{G, E, S \vdash e_1 : v_1, S_1 \qquad G, E, S_1 \vdash e_2 : v_2, S_2}{G, E, S \vdash e_1 < e_2 : (v_1 < v_2), S_2}\ \text{E-Lt}$$

subscripts

e_1, its value v_1, the store it leaves S_1

wrap

keep the result in 32 bits

The second premise starts from S_1: the left operand runs first.

Calling a function.

$$\frac{G, E, S \vdash e : v, S_1 \quad \ell \text{ fresh} \quad G, \{a \mapsto \ell\}, S_1[\ell \mapsto v] \vdash \text{body} : S_2, r}{G, E, S \vdash f(e) : r, S_2} \text{ E-Call}$$

(f has one parameter, a)

$$\frac{G, E, S \vdash e : v, S_1}{G, E, S \vdash \text{return } e : S_1, v} \text{ S-Return}$$

fresh location

a location no one uses yet: each call gets its own a

`return` puts its value in `R'`. The call takes it out as its value.

`if` and `while`.

$$\frac{G, E, S \vdash c : \texttt{True}, S_1 \qquad G, E, S_1 \vdash s_1 \ldots s_n : S_2, R'}{G, E, S \vdash \texttt{if } c\texttt{: } s_1 \ldots s_n \texttt{ else: } t_1 \ldots t_m : S_2, R'}\ \text{S-IfTrue} \qquad (\text{S-IfFalse: run } t_1 \ldots t_m)$$

$$\frac{G, E, S \vdash c : \texttt{False}, S_1}{G, E, S \vdash \texttt{while } c\texttt{: } s : S_1, \texttt{None}}\ \text{S-WhileFalse}$$

$$\frac{G, E, S \vdash c : \texttt{True}, S_1 \qquad G, E, S_1 \vdash s : S_2, \texttt{None} \qquad G, E, S_2 \vdash \texttt{while } c\texttt{: } s : S_3, R'}{G, E, S \vdash \texttt{while } c\texttt{: } s : S_3, R'}\ \text{S-WhileTrue}$$

`if` runs one branch. The type rule checked both.

`while` uses itself as a premise: that is the loop.

`print` and runtime errors.

$$\frac{G, E, S \vdash e : v, S_1}{G, E, S \vdash \texttt{print}(e) : S_1 + v, \text{None}}\ \text{E-Print}$$

$$\frac{G, E, S \vdash e_1 : v_1, S_1 \qquad G, E, S_1 \vdash e_2 : 0, S_2}{G, E, S \vdash e_1 \texttt{ // } e_2 : \textbf{error}, S_2}\ \text{E-DivZero}$$

`S + v`
the store with one more line of output

`error`
the program stops, exit status 1

A runtime error is a rule, not a crash. The type checker accepts `10 // 0`.

Example: `inc(x)`.

run.py · from Lecture 9

```
x: int = 3
def inc(a: int) -> int:
    return a + 1
print(inc(x) < 5)
```

Before line 4: $G = \{x \mapsto \ell_x\}$, $S = \{\ell_x \mapsto 3\}$, $E = \{\}$

Inside the call: $E_a = \{a \mapsto \ell_a\}$, $S_a = S[\ell_a \mapsto 3]$

$$\dfrac{\dfrac{}{G, E_a, S_a \vdash a : 3, S_a}\ \text{E-Var} \quad \dfrac{}{G, E_a, S_a \vdash 1 : 1, S_a}\ \text{E-Const}}{\dfrac{G, E_a, S_a \vdash a + 1 : 4, S_a}{G, E_a, S_a \vdash \texttt{return a + 1} : S_a, 4}\ \text{S-Return}}\ \text{E-Add}$$

$$\dfrac{\dfrac{}{G, E, S \vdash x : 3, S}\ \text{E-Var} \qquad \cdots}{G, E, S \vdash \texttt{inc(x)} : 4, S_a}\ \text{E-Call}$$

Lecture 9's tree said `int` here. This one says 4.

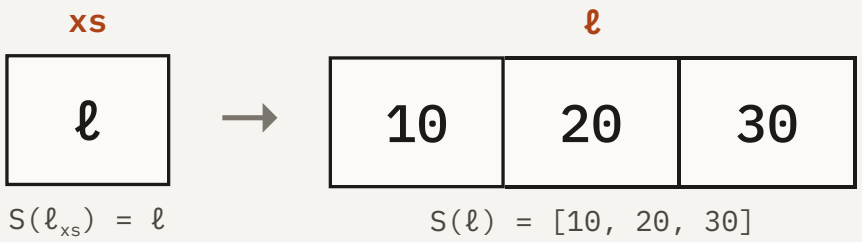
Example: the rest of line 4.

$$\dfrac{G, E, S \vdash \texttt{inc(x)} : 4, S_a \quad \text{(slide 8)} \qquad G, E, S_a \vdash 5 : 5, S_a}{G, E, S \vdash \texttt{inc(x)} < 5 : \texttt{True}, S_a}\ \text{E-Lt}$$

$$\dfrac{G, E, S \vdash \texttt{inc(x)} < 5 : \texttt{True}, S_a}{G, E, S \vdash \texttt{print(inc(x)} < 5) : S_a + \texttt{True}, \texttt{None}}\ \text{E-Print}$$

The program's meaning is its output: `True`.

A list value is a location.



ℓ
a location in the store

A variable holds the location, not the list.

None is a value too: no list at all.

Typing rules.

$$\frac{\Gamma \vdash e_1 : T \quad \ldots \quad \Gamma \vdash e_n : T \quad (n \geq 1)}{\Gamma \vdash [e_1, \ldots, e_n] : [T]}\ \text{[list-display]}$$

$$\frac{\Gamma \vdash e_1 : [T] \qquad \Gamma \vdash e_2 : \texttt{int}}{\Gamma \vdash e_1[e_2] : T}\ \text{[list-select]}$$

$$\frac{\Gamma \vdash e : [T]}{\Gamma \vdash \texttt{len}(e) : \texttt{int}}\ \text{[len]}$$

$$\frac{\Gamma \vdash e_1 : [T] \qquad \Gamma \vdash e_2 : \texttt{int} \qquad \Gamma \vdash e_3 : T}{\Gamma, R \vdash e_1[e_2] = e_3}\ \text{[list-assign-stmt]}$$

`[T]`
a list of `T`s, e.g. `[int]`

All elements share one type. The index is an `int`.

[] and None.

```
xs: [int] = []        # ok
xs: [int] = None      # ok: a list may be None
n:  int   = None      # error: expected int, found None
```

the manual

§2.4 writes this as a relation, $\leq_a$. It accepts the same programs.

These two have no type of their own. They take the type of the variable they're stored into.

Evaluation rules.

$$\frac{G, E, S \vdash e_1 : v_1, S_1 \quad \dots \quad G, E, S_{n-1} \vdash e_n : v_n, S_n \qquad \ell \text{ fresh}}{G, E, S \vdash [e_1, \dots, e_n] : \ell, S_n[\ell \mapsto [v_1, \dots, v_n]]}\ \text{E-List}$$

$$\frac{G, E, S \vdash e_1 : \ell, S_1 \qquad G, E, S_1 \vdash e_2 : k, S_2 \qquad S_2(\ell) = [v_0, \dots, v_{n-1}] \qquad 0 \le k < n}{G, E, S \vdash e_1[e_2] : v_k, S_2}\ \text{E-Index}$$

$$\frac{G, E, S \vdash e : \ell, S_1 \qquad S_1(\ell) = [v_0, \dots, v_{n-1}]}{G, E, S \vdash \text{len}(e) : n, S_1}\ \text{E-Len}$$

$$\frac{G, E, S \vdash e_1 : \ell, S_1 \qquad G, E, S_1 \vdash e_2 : k, S_2 \qquad G, E, S_2 \vdash e_3 : v, S_3 \qquad 0 \le k < n}{G, E, S \vdash e_1[e_2] = e_3 : S_3[\ell[k] \mapsto v], \text{None}}\ \text{S-ListAssign}$$

$S[\ell[k] \mapsto v]$

the list at ℓ, with element k replaced by v

A list display makes a new list. Indexing reads it; assignment changes it in place.

Errors for lists.

$$\frac{G, E, S \vdash e_1 : \ell, S_1 \quad G, E, S_1 \vdash e_2 : k, S_2 \quad S_2(\ell) \text{ has } n \text{ elements} \quad k < 0 \text{ or } k \geq n}{G, E, S \vdash e_1[e_2] : \textbf{error}, S_2}\ \text{E-IndexError}$$

$$\frac{G, E, S \vdash e_1 : \text{None}, S_1}{G, E, S \vdash e_1[e_2] : \textbf{error}, S_1}\ \text{E-IndexNone}$$

(also for len and list assignment)

The type checker can't see either: `i : int` says nothing about its value.

Each has a rule, so each is a defined error.

Example: two names, one list.

alias.py · prints 9

```
xs: [int] = [10, 20]
ys: [int] = xs
ys[0] = 9
print(xs[0])
```

the store after each line

after line	$S(\ell_{xs})$	$S(\ell_{ys})$	$S(\ell)$
1	ℓ		[10, 20]
2	ℓ	ℓ	[10, 20]
3	ℓ	ℓ	[9, 20]

$$\frac{}{G, E, S \vdash xs : \ell, S}\ \text{E-Var} \qquad \frac{}{G, E, S \vdash 0 : 0, S}\ \text{E-Const} \qquad S(\ell) = [9, 20]$$

$$\frac{G, E, S \vdash xs : \ell, S \qquad G, E, S \vdash 0 : 0, S \qquad S(\ell) = [9, 20]}{G, E, S \vdash xs[0] : 9, S}\ \text{E-Index}$$

$$\frac{G, E, S \vdash xs[0] : 9, S}{G, E, S \vdash \text{print}(xs[0]) : S + 9, \text{None}}\ \text{E-Print}$$

Line 2 copies the location, not the list. So changing ys changes xs.

Types meet values

Well-typed programs don’t get stuck.

A run ends one of three ways: a value of the predicted type, a named `error`, or never.

If the program type-checks, no fourth way is possible.

stuck

no rule applies. `4 < True` is stuck: `E-Lt` needs two integers. The type checker rejects it first.

PA4.

PA4, due Fri 11/13: lists and strings. Slides 11–12 are the checker; slides 13–14 say what the output must be.

Thursday's section is the PA4 lab. No lecture Wed 11/11.